



Lab5

简单的类 MIPS 单周期 CPU

CS2306

姓名: 吴恒涛

学号: 524031910038

日期: 2026 年 6 月 2 日



摘要

本次实验完成了一个简单的类 MIPS 单周期 CPU 的设计、仿真与上板验证。实验首先根据单周期处理器的数据通路完成 PC、指令存储器、寄存器组、ALU、数据存储器以及控制模块之间的连接，并实现 lw, sw, beq, add, sub, and, or, slt, j 等 9 条基本指令。在此基础上，又进一步扩展 addi, andi, ori, sll, srl, jal, jr 等 7 条指令，使处理器能够支持共 16 条指令。最后通过乘法程序进行功能仿真，并将程序下载到实验板上，使用拨码开关和七段数码管完成结果验证。

目录

1 实验目标	2
2 原理分析	2
2.1 单周期 CPU 整体结构	2
2.2 关键模块说明	2
3 功能具体实现	3
3.1 PC 模块	3
3.2 指令存储器	4
3.3 数据通路连接	4
3.4 PC 更新逻辑	5
3.5 寄存器组复位与写回	5
3.6 立即数扩展	6
3.7 控制信号设计	6
4 功能仿真	7
4.1 乘法程序说明	7
4.2 乘法程序仿真结果	8
5 上板验证	9
5.1 上板前的修改	9
5.2 乘法器上板结果	10
6 结果分析	11
7 总结反思	11



1 实验目标

本实验的主要目标如下：

1. 理解简单的类 MIPS 单周期处理器的基本工作原理，包括指令执行过程中所需的数据通路、控制信号以及各功能模块之间的连接关系。
2. 完成简单的类 MIPS 单周期 CPU 的设计与调试：
 - (a) 实现 lw, sw, beq, add, sub, and, or, slt, j 共 9 条基本指令；
 - (b) 在 9 条指令的基础上继续扩展 addi, andi, ori, sll, srl, jal, jr, 共实现 16 条指令。
3. 编写测试程序并在 Vivado 中进行功能仿真。
4. 将设计下载到实验开发板上，通过拨码开关和七段数码管完成上板验证。

2 原理分析

2.1 单周期 CPU 整体结构

单周期 CPU 的特点是每条指令都在一个时钟周期内完成，因此取指、译码、执行、访存和写回这些步骤需要在同一个周期中通过组合逻辑完成。这样设计的优点是结构较为直接，便于观察不同模块之间的数据流向；缺点是时钟周期需要由最慢的指令决定，因此性能上并不是最优。

本实验中的类 MIPS 单周期 CPU 主要由以下几个部分组成：程序计数器 PC、指令存储器 Instruction Memory、寄存器组 Registers、算术逻辑单元 ALU、数据存储器 Data Memory、主控制器 Control 以及 ALU 控制器 ALU Control。处理器运行时，PC 给出当前指令地址，指令存储器根据地址输出指令，控制器根据指令 opcode 生成对应控制信号，随后数据沿着寄存器组、ALU、数据存储器和写回通路完成一次指令执行。

如图 1 所示，主控制器会产生 RegDst、Jump、Branch、MemRead、MemtoReg、ALUOp、MemWrite、ALUSrc 和 RegWrite 等信号。这些信号分别控制寄存器写入目标、PC 跳转方式、是否访存、ALU 操作数来源以及写回数据来源。ALU Control 则根据 ALUOp 和 funct 字段进一步决定 ALU 的具体运算类型。

2.2 关键模块说明

除顶层模块外，数据通路中还包含 Ctr、ALUCtr、ALU、DataMemory、Signext、PC、InstructionMemory 等模块。其中 PC 和指令存储器是本次实验需要重点补充的部分。

PC 本质上是一个 32 位寄存器，用于保存当前指令地址。每个时钟周期到来时，PC 根据当前指令类型选择更新为 PC+4、分支地址、跳转地址或寄存器中的地址。指令存

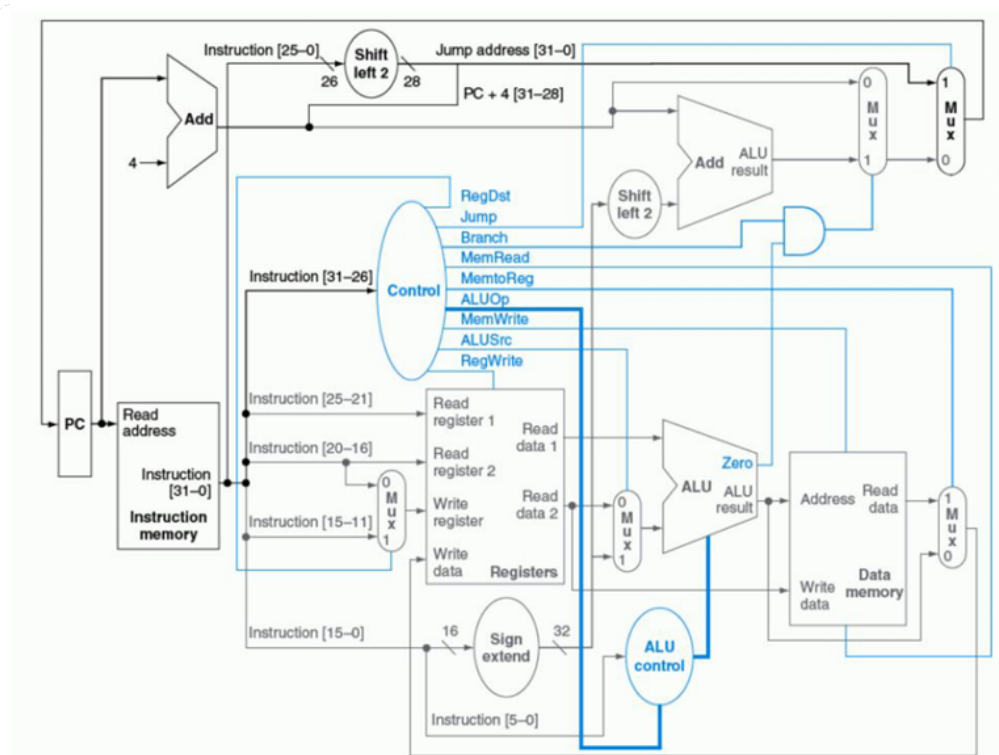


图 1: 单周期 CPU 数据通路结构图

存储器用于根据 PC 输出对应的 32 位机器指令。由于本实验中指令内容在运行时不需要被修改，因此指令存储器可以设计成只读形式。

寄存器组负责提供两个读端口和一个写端口。对于 R 型指令，两个源操作数通常来自寄存器；对于 I 型指令，第二个操作数可能来自立即数扩展模块。ALU 负责完成加法、减法、逻辑与、逻辑或、比较、移位等运算。数据存储器主要服务于 lw 和 sw 指令，用于完成内存读写。

3 功能具体实现

3.1 PC 模块

PC 模块用于保存和更新当前指令地址。reset 有效时，PC 被清零；否则在时钟上升沿更新为输入的新地址。实现代码如下：

```
module PC(
    input [31:0] in,
    input reset,
    input clk,
    output reg [31:0] out
);
    always @(posedge clk) begin
```



```
if (reset)
    out <= 32'b0;
else
    out <= in;
end
endmodule
```

这里使用非阻塞赋值更新 PC，保证其综合后对应时序寄存器。PC 的输入并不是固定的 PC+4，而是由顶层模块中的选择逻辑决定。

3.2 指令存储器

指令存储器根据 PC 给出的地址输出对应指令。由于 MIPS 指令按字节编址，而实验中的存储数组按 32 位字存储，因此访问时需要使用地址的高位，或者将地址右移两位。典型实现如下：

```
module InstructionMemory(
    input [31:0] address,
    output [31:0] instruction
);
    reg [31:0] instrMEM [0:63];

    initial begin
        $readmemb("Instruction.txt", instrMEM);
    end

    assign instruction = instrMEM[address >> 2];
endmodule
```

这样可以使 PC 按照 0、4、8、12 的字节地址递增时，正确对应到指令存储器中的第 0、1、2、3 条指令。

3.3 数据通路连接

在顶层模块中，需要把 PC、InstructionMemory、Registers、ALU、DataMemory、Ctr 和 ALUCtr 等模块实例化并连接起来。对于普通连线，可以直接使用同名 wire；对于需要选择的数据，则使用三目运算符构造多路选择器。

以 ALU 输入选择为例：

```
assign input1 = aluSrc1 ? inst[10:6] : readData1;
assign input2 = aluSrc ? imm : readData2;

ALU alu (
    .input1(input1),
    .input2(input2),
```



```
.ALUctrOut(aluCtrOut),  
.ALUres(aluRes),  
.zero(zero)  
);
```

其中 input1 的选择主要用于支持 sll 和 srl 指令，因为这两条指令需要使用 shamt 字段作为移位量。input2 的选择用于区分 R 型指令和 I 型指令，当 aluSrc 为 1 时，第二个 ALU 操作数来自立即数扩展结果。

3.4 PC 更新逻辑

PC 的更新逻辑是整个单周期 CPU 中比较关键的部分。普通指令执行后 PC 更新为 PC+4；beq 成立时 PC 更新为分支目标地址；j 和 jal 需要跳转到指令中给出的目标地址；jr 则需要跳转到寄存器保存的地址。实现方式如下：

```
assign pcplus4 = pc + 4;  
  
assign newpc = jump[0] ? {pcplus4[31:28], inst[25:0], 2'b0}  
               : (jump[1] ? readData1  
               : (branch && zero ? (imm << 2) + pcplus4 : pcplus4));
```

这里 jump 被扩展为两位：jump[0] 用于普通 j 和 jal 跳转，jump[1] 用于 jr 指令。beq 指令通过 branch 和 zero 共同决定是否发生跳转。

3.5 寄存器组复位与写回

为了保证 CPU 从确定状态开始运行，reset 时需要清空寄存器组。寄存器写入一般放在时钟边沿触发的 always 块中完成：

```
integer i;  
  
always @(negedge clk) begin  
    if (reset) begin  
        for (i = 0; i < 32; i = i + 1)  
            regFile[i] <= 32'b0;  
        end  
    else if (regWrite) begin  
        regFile[writeReg] <= writeData;  
    end  
end
```

写回目标寄存器和写回数据都需要通过控制信号选择。对于普通 R 型指令，目标寄存器为 rd；对于 I 型指令，目标寄存器为 rt；对于 jal 指令，需要将返回地址写入 31 号寄存器。



```
assign writeReg = regDst[1] ? 5'd31 :  
                  (regDst[0] ? inst[15:11] : inst[20:16]);  
  
assign writeData = regDst[1] ? pcplus4 :  
                  (memToReg ? readData : aluRes);
```

3.6 立即数扩展

不同 I 型指令对立即数扩展的要求并不完全相同。addi、lw、sw、beq 等指令需要符号扩展；andi 和 ori 等逻辑立即数指令更适合使用零扩展。因此我增加 extOp 控制信号，用于选择扩展方式：

```
module Signext(  
    input extOp,  
    input [15:0] inst,  
    output [31:0] data  
);  
    assign data = extOp ? {{16{inst[15]}}, inst}  
                      : {{16{1'b0}}, inst};  
endmodule
```

当 extOp 为 1 时，执行符号扩展；当 extOp 为 0 时，执行零扩展。

3.7 控制信号设计

在完成基本数据通路之后，控制信号的设计决定了每条指令在同一套硬件结构中具体走哪一条路径。由于单周期 CPU 中所有功能都在一个周期内完成，因此控制信号需要在指令译码后立即确定，包括寄存器是否写入、ALU 的输入来源、数据存储器是否读写、写回数据来自 ALU 还是内存，以及 PC 是否发生跳转等。

在最初的 9 条基本指令中，控制信号的设计相对直接。例如，R-type 指令需要从寄存器组读出两个操作数，经过 ALU 运算后写回 rd；lw 指令需要用 ALU 计算访存地址，再从 Data Memory 读出数据写回寄存器；sw 指令同样需要计算访存地址，但只写数据存储器而不写寄存器；beq 指令需要比较两个寄存器的值，并在 zero 信号有效时改变 PC；j 指令则直接使用指令中的跳转目标地址更新 PC。

在扩展到 16 条指令之后，原有控制信号需要做一定修改。首先，regDst 从 1 位扩展为 2 位，用于区分写入 rt、rd 以及 31 号寄存器三种情况，其中 jal 指令需要将返回地址写入 \$31。其次，Jump 也扩展为 2 位，用于区分普通的 j/jal 跳转和 jr 指令的寄存器跳转。再次，ALUOp 扩展为 3 位，使控制器能够进一步区分 addi、andi、ori 等立即数指令。最后，ExtOp 用于选择立即数扩展方式，其中 addi、lw、sw 等指令使用符号扩展，而 andi、ori 使用零扩展。



为了使表格更符合信号功能的理解顺序，下面将控制信号按“寄存器写回、ALU 控制、PC 控制、存储器访问、立即数扩展”的顺序排列。各指令对应的主要控制信号如表 1 所示。

表 1: 主要控制信号表

指令	regDst	regWrite	ALUsrc	ALUop	Branch	Jump	memRead	memWrite	MemToReg	Extop
R-type	01	1	0	010	0	00	0	0	0	X
lw	00	1	1	000	0	00	1	0	1	1
sw	00	0	1	000	0	00	0	1	0	1
beq	00	0	0	001	1	00	0	0	0	X
j	X	0	X	X	X	01	X	0	X	X
addi	00	1	1	000	0	00	0	0	0	1
andi	00	1	1	100	0	00	0	0	0	0
ori	00	1	1	101	0	00	0	0	0	0
sll/srl	01	1	0	010	0	00	0	0	0	X
jal	10	1	0	000	0	01	0	0	0	X
jr	X	0	X	X	X	10	X	0	X	X

从表中可以看出，不同指令虽然共用同一套硬件模块，但控制信号组合并不相同。比如 lw 和 sw 都需要 ALU 计算地址，所以它们的 ALUsrc 都为 1，ALUop 都选择加法；但 lw 需要读存储器并写回寄存器，所以 memRead 和 regWrite 为 1，而 sw 只写存储器，因此 memWrite 为 1、regWrite 为 0。R-type、sll 和 srl 都需要写回寄存器，但移位指令还需要额外通过 aluSrc1 选择 shamt 字段作为移位量，这一点没有直接体现在主控制信号表中，而是在 ALU 输入选择逻辑中完成。

jal 和 jr 是扩展跳转指令中最需要特别处理的两类。jal 一方面要跳转，另一方面还要把 PC+4 写入 31 号寄存器，因此它的 Jump 为 01，regDst 为 10，regWrite 为 1。jr 则不使用指令中的立即跳转地址，而是直接把寄存器读出的值作为新的 PC，因此 Jump 设置为 10，并且不需要写回寄存器。通过这些控制信号的区分，扩展后的 16 条指令就可以在同一套单周期数据通路上完成执行。

4 功能仿真

4.1 乘法程序说明

为了验证处理器功能，我使用乘法程序作为测试程序。该程序通过移位和累加实现两个数的乘法。测试数据为 9 和 13，理论结果为：

$$9 \times 13 = 117$$

十进制 117 对应十六进制为：

$$117 = 0x75$$



测试汇编代码大致如下:

```
lw  $1, 0($3)
lw  $2, 4($3)
srl $4, $2, 1
sll $5, $4, 1
beq $5, $2, 1
add $8, $8, $1
srl $2, $2, 1
sll $1, $1, 1
beq $2, $3, 1
j   2
sw  $8, 8($3)
```

程序首先从数据存储器中读入两个乘数, 然后通过判断乘数最低位是否为 1 来决定是否将当前被乘数累加到结果寄存器中。随后乘数右移、被乘数左移, 循环直到乘数为 0。最后使用 sw 指令将结果写回数据存储器。

4.2 乘法程序仿真结果

完成数据通路和控制信号连接后, 我在 Vivado 中进行了行为仿真。仿真时重点观察 PC、inst、寄存器组、memWrite、ALU 输出、写回数据以及数据存储器中的关键位置。

图 2 展示了乘法程序后半段的仿真波形。可以看到, 随着时钟推进, 寄存器中的数值按照乘法程序逻辑逐步更新。其中 Reg[1] 和 Reg[2] 分别参与移位操作, Reg[8] 用于保存累加结果。最终 Reg[8] 的值变为十六进制 00000075, 即十进制 117, 与 9×13 的理论结果一致。同时, 数据存储器中对应位置也被写入 00000075, 说明 sw 指令能够正确完成写存储器操作。

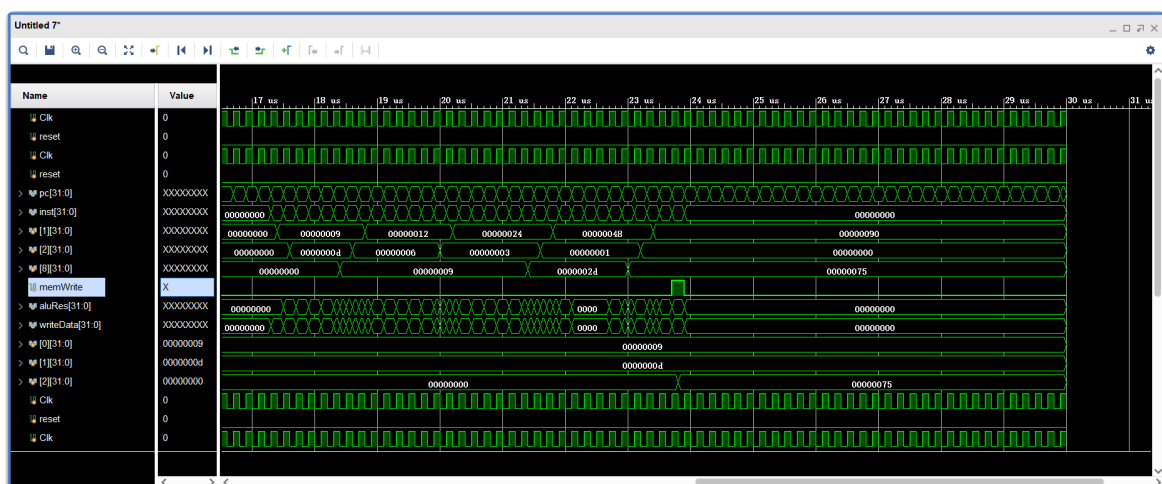


图 2: 乘法程序仿真结果

图 3 展示了从仿真开始阶段观察到的波形。reset 结束后, PC 开始按照指令执行流程变化, 指令存储器不断输出新的机器码, 寄存器组中的值也随指令运行逐步改变。可



以看到，初始数据 9 和 13 被正确读入，随后程序通过移位、比较、条件分支和累加等操作完成乘法计算。该波形说明 lw、sll、srl、beq、add、j、sw 等指令之间的数据流转能够正常配合，控制信号也能够根据不同指令类型正确切换。

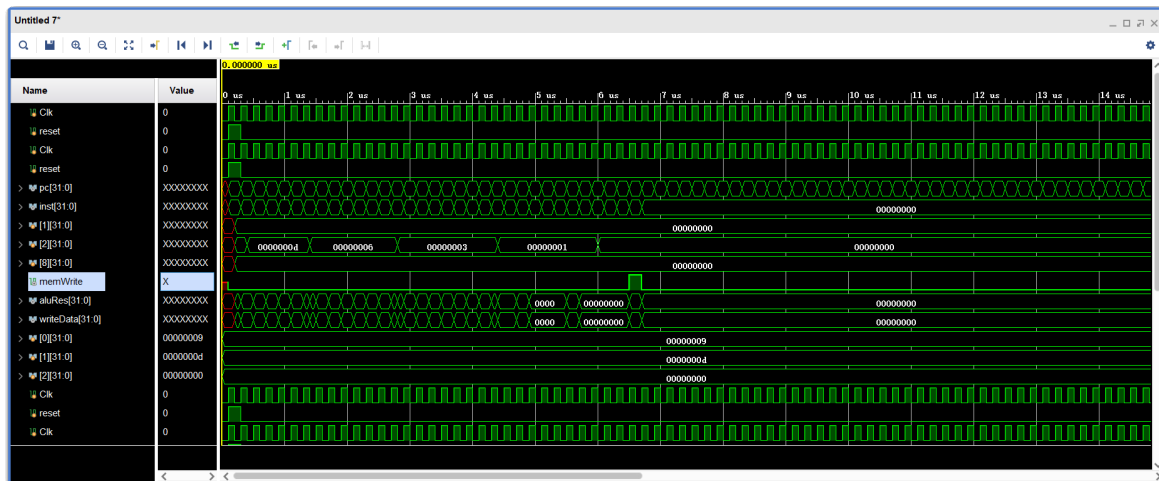


图 3: 乘法程序初始阶段仿真波形

5 上板验证

5.1 上板前的修改

仿真通过后，我将设计下载到实验开发板上进行验证。与仿真环境不同，上板时不能依赖 initial 块完成初始化，因此需要将指令存储器和数据存储器的初始化逻辑改写到 reset 分支中。同时，为了能够直接观察输入和输出结果，我将拨码开关、LED 和七段数码管显示模块接入顶层模块。

其中，时钟和 reset 的处理大致如下：

```
wire CLK_i;
wire Clk_25M;

IBUFGDS IBUFGDS_inst (
    .O(CLK_i),
    .I(clk_p),
    .IB(clk_n)
);

reg [1:0] clkdiv;
always @(posedge CLK_i)
    clkdiv <= clkdiv + 1;

assign Clk_25M = clkdiv[1];
```

```
wire Clk;  
wire reset;  
  
assign Clk = Clk_25M;  
assign reset = !_reset;
```

这里通过分频得到较低频率的时钟信号，便于开发板上的程序稳定运行。由于板上 reset 按键是低电平有效，因此需要使用 `assign reset = !_reset;` 进行转换。

七段数码管显示部分用于显示输入乘数和计算结果。输入数据通过拨码开关给出，CPU 运行结束后，结果会被写入数据存储器，并进一步送到显示模块中观察。

5.2 乘法器上板结果

图 4 为上板验证结果。拨码开关用于设置两个输入乘数，程序运行结束后，七段数码管显示计算出结果。图中可以看到，输入为 7 和 5，输出结果为 23。这里的 23 是十六进制显示，对应十进制 35，正好满足：

$$7 \times 5 = 35 = 0x23$$

因此，上板结果与预期一致，说明处理器不仅在仿真环境下能够正确执行测试程序，在实际 FPGA 板上也能够完成取指、译码、执行、访存和写回等过程。

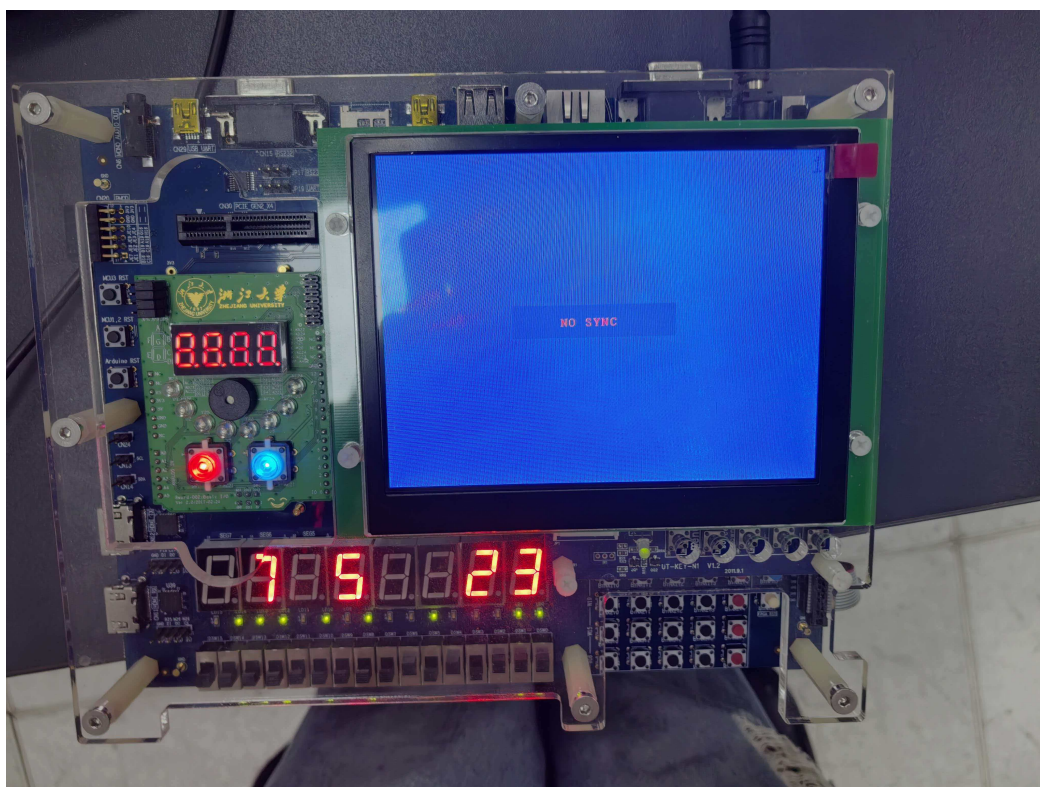


图 4: 乘法程序上板验证结果



6 结果分析

通过仿真和上板结果可以看出，本次设计的类 MIPS 单周期 CPU 能够正确执行测试程序。对于乘法程序而言，计算过程涉及多种指令类型，包括访存指令 `lw` 和 `sw`、算术指令 `add`、移位指令 `sll` 和 `srl`、条件分支指令 `beq` 以及无条件跳转指令 `j`。这些指令覆盖了数据通路中的主要功能模块，因此可以比较全面地验证处理器设计是否正确。

从仿真波形来看，寄存器中的中间值变化符合乘法程序的执行逻辑，最终结果也被正确写入数据存储器。从上板结果来看，拨码开关输入和七段数码管输出能够对应，进一步说明硬件实现与仿真结果一致。

本次实验中较容易出错的部分主要有以下几点：第一，指令和数据存储器采用字寻址，而 PC 和 MIPS 指令格式中使用的是字节地址，因此访问存储器时需要处理地址低两位；第二，`reset` 逻辑需要保证 PC、寄存器组和存储器中的关键数据处于确定状态；第三，`jal` 和 `jr` 这类跳转指令需要修改原有控制信号位宽，否则普通 `jump` 逻辑无法覆盖所有情况；第四，上板时不能简单依赖仿真中的 `initial` 初始化方式，需要针对 FPGA 实际运行方式重新处理初始化。

7 总结反思

本次实验从单周期 CPU 的基本数据通路出发，逐步完成了 9 条基本 MIPS 指令以及 7 条扩展指令的设计。通过这一过程，我对类 MIPS 单周期处理器的结构有了更加直观的理解，也进一步熟悉了 Verilog 中组合逻辑、时序逻辑、模块实例化和控制信号设计的方法。

实验中最核心的部分是让数据通路和控制信号严格对应。不同指令虽然共享同一套硬件结构，但它们使用的路径并不相同。例如，R 型指令需要从寄存器组读取两个操作数并写回 `rd`；`lw` 指令需要经过数据存储器并将读出的数据写回寄存器；`sw` 指令只写存储器而不写寄存器；`beq` 和 `j` 指令则需要改变 PC 的更新路径。扩展到 16 条指令后，还要额外考虑立即数扩展、移位量选择、`jal` 的返回地址写入以及 `jr` 的寄存器跳转等情况。

最终的仿真结果和上板结果均与预期一致，说明本次 CPU 设计能够正确执行测试程序。通过本次实验，我不仅理解了单周期 CPU 中“取指—译码—执行—访存—写回”的完整流程，也体会到硬件设计中时钟、`reset`、初始化和外设显示等细节对实验结果的重要影响。